Was ist Vagrant? – Grundlagen

Schritt für Schritt vagramt Einstieg ohne Vorkenntnisse

Michael Schaar

26. Oktober 2026

Inhaltsverzeichnis

Was ist Vagrant? – Grundlagen **2**

1. Das Ausgangsproblem 2
2. Was ist Vagrant? 2
3. Kernkonzepte im Überblick 3
4. Typischer Aufbau eines `Vagrantfile` 4
5. Die wichtigsten Befehle 5
6. Typische Fehlerquellen und Grenzen 6

Was ist Vagrant? – Grundlagen

Diese Dokumentation erklärt die Funktionsweise von Vagrant und dessen Rolle in diesem Projekt.

---

1. Das Ausgangsproblem

Robotik-Software wie das Robot Operating System (ROS), Simulatoren (Gazebo) und hardwarenahe Bibliotheken stellen hohe Anforderungen an das Betriebssystem. Häufig wird eine ganz bestimmte Linux-Distribution oder Kernel-Version vorausgesetzt.

Wer eine solche Umgebung manuell aufsetzt, stößt schnell auf Probleme:

- **Unterschiedliche Host-Systeme:** Im Team laufen Windows, macOS und verschiedene Linux-Distributionen.
- **Hoher Einrichtungsaufwand:** Das manuelle Installieren von Paketen, Umgebungsvariablen und Treibern dauert Stunden und ist fehleranfällig.
- **Konfigurationsdrift:** Kleine Abweichungen („Auf meinem Rechner lief es gestern aber noch“) erschweren die Fehlersuche im Laborbetrieb.

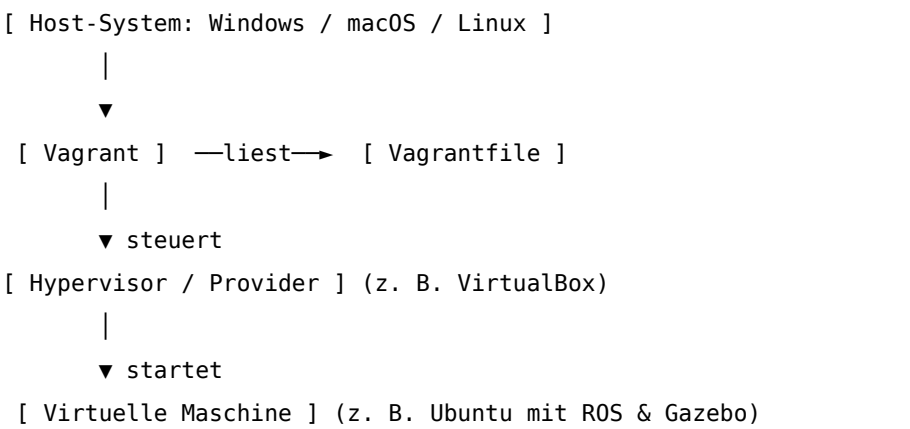
---

2. Was ist Vagrant?

Vagrant ist ein quelloffenes Kommandozeilen-Werkzeug zur Automatisierung und Steuerung virtueller Maschinen (VMs).

Wichtig dabei: **Vagrant ist kein eigener Hypervisor.** Es erzeugt die virtuelle Maschine nicht selbst, sondern steuert vorhandene Virtualisierungssoftware wie VirtualBox, VMware oder Libvirt fern.

Vagrant beschreibt die gesamte Umgebung in einer einzigen Textdatei: dem `Vagrantfile`. Dadurch lässt sich eine komplette Entwicklungsumgebung versionieren, über Git teilen und auf jedem Zielrechner mit einem einzigen Befehl identisch aufbauen.



---

3. Kernkonzepte im Überblick

Um mit Vagrant zu arbeiten, genügen vier Grundbegriffe:

Provider

Die Virtualisierungs-Engine, die die VM ausführt. In diesem Projekt ist das standardmäßig **VirtualBox**. Vagrant übernimmt die Konfiguration von CPU, RAM und Netzwerkkarten direkt beim Provider.

Box

Das Basis-Image eines Betriebssystems (vergleichbar mit einem Docker-Image oder einem ISO-Installationsabbild). Anstatt ein Betriebssystem interaktiv über ein Installationsmenü aufzusetzen, lädt Vagrant eine vorgefertigte Box herunter (meist von der Plattform *Vagrant Cloud*) und klont sie lokal.

Vagrantfile

Die zentrale Steuerungsdatei im Projektordner. Sie ist in Ruby-Syntax verfasst und definiert:

- welche Basis-Box genutzt wird
- wie viele Ressourcen (RAM, CPU-Kerne) die VM erhält

- welche Verzeichnisse zwischen Host und VM synchronisiert werden
- welche Installationsschritte beim ersten Start ausgeführt werden

Provisioning (Bereitstellung)

Das automatisierte Einrichten der Software *nach* dem ersten Hochfahren des Betriebssystems. Über Shell-Skripte oder Konfigurationswerkzeuge (wie Ansible) installiert Vagrant automatisch ROS, Entwicklerwerkzeuge und Abhängigkeiten, ohne dass manuelle Eingaben nötig sind.

---

4. Typischer Aufbau eines `Vagrantfile`

Ein auf dieses Labor zugeschnittenes `Vagrantfile` sieht im Kern wie folgt aus:

```
Vagrant.configure("2") do |config|
  # 1. Basis-Image wählen (z. B. Ubuntu 22.04 LTS)
  config.vm.box     = "ubuntu-2404-desktop"
  config.vm.box_url = "https://hel1.your-objectstorage.com/kraeml-bayern/ubuntu-2404-
desktop/metadata.json"

  # 2. Ressourcen und GUI für Robotik-Werkzeuge festlegen
  config.vm.provider "virtualbox" do |vb|
    vb.gui = true               # Grafisches Fenster für Gazebo / RViz öffnen
    vb.memory = "4096"          # Mindestens 4 GB Arbeitsspeicher
    vb.cpus = 2                 # Mindestens 2 CPU-Kerne
    vb.name = "robotics-lab-vm"
  end

  # 3. Synchronisierte Ordner (Projektdateien auf dem Host bearbeiten)
  config.vm.synced_folder ".", "/home/vagrant/shared"

  # 4. Provisioning: Automatisches Installationsskript ausführen
  config.vm.provision "shell", path: "setup.sh"
end
```

Geteilte Verzeichnisse (Synced Folders)

Dateien im Projektordner des Hosts stehen standardmäßig innerhalb der VM unter `/vagrant` (oder dem konfigurierten Zielpfad) zur Verfügung.

Der Vorteil: Man schreibt Code mit dem gewohnten Editor auf dem Host-Rechner (z. B. VS Code), kompiliert und testet diesen aber innerhalb der Linux-VM.

---

5. Die wichtigsten Befehle

Alle Befehle werden im Terminal innerhalb des Projektverzeichnisses ausgeführt, in dem die Datei `Vagrantfile` liegt:

Befehl	Funktion
`vagrant up`	Erstellt und startet die VM. Beim ersten Aufruf wird die Box heruntergeladen und das Provisioning ausgeführt.
`vagrant halt`	Fährt das Gastsystem sauber herunter.
`vagrant suspend`	Friert den aktuellen Zustand der VM ein (spart Zeit beim nächsten Start).
`vagrant resume`	Weckt eine pausierte VM wieder auf.
`vagrant ssh`	Öffnet eine SSH-Sitzung direkt in das Gastsystem.
`vagrant provision`	Führt die Provisioning-Skripte erneut aus, ohne die VM neu zu erstellen.
`vagrant reload`	Startet die VM neu (z. B. nötig nach Änderungen am `Vagrantfile`).
`vagrant reload --provision`	Startet die VM neu und erzwingt das Ausführen der Installationsskripte.
`vagrant destroy`	Löscht die VM mitsamt ihrer virtuellen Festplatte vollständig. Das Projektverzeichnis auf dem Host bleibt erhalten.

---

6. Typische Fehlerquellen und Grenzen

Hardware-Virtualisierung im BIOS/UEFI deaktiviert

- **Symptom:** `vagrant up` bricht mit Fehlermeldungen wie `VT-x is not available` oder `VERR_VMX_NO_VMX` ab.
- **Lösung:** Virtualisierung (Intel VT-x bzw. AMD-V) im BIOS/UEFI des Rechners aktivieren.

Hyper-V-Konflikte (unter Windows)

- **Symptom:** VirtualBox stürzt ab oder startet die 64-Bit-Instanz extrem langsam.
- **Ursache:** VirtualBox und Windows-eigene Hyper-V-Funktionen (einschließlich WSL2 oder Memory Integrity) blockieren sich mitunter gegenseitig.

Apple Silicon (M1/M2/M3/M4)

- **Problem:** Ältere `Vagrantfile`-Konfigurationen basieren oft fest auf `x86_64`-Boxen und VirtualBox. VirtualBox unterstützt ARM-Architekturen auf macOS nur eingeschränkt.
- **Lösung:** Falls ein Apple-Silicon-Rechner genutzt wird, müssen angepasste ARM64-Boxen und passende Provider (wie QEMU/UTM oder VMware Fusion Tech Preview) verwendet werden.

Ressourcenmangel bei 3D-Simulationen

- ROS-Tools wie RViz oder Simulationsumgebungen wie Gazebo benötigen Grafikbeschleunigung und CPU-Leistung. Weist das Hostsystem der VM zu wenig RAM (unter 4 GB) oder nur einen CPU-Kern zu, frieren grafische Anwendungen häufig ein.